

Architecture Distribuée

&

Cloud Computing

Cours & Travaux Pratiques

Dr.BOUKIL

Architecture Distribuée et Informatique en Nuage

Objectifs:

- Comprendre les principes fondamentaux des systèmes distribués.
- Identifier et comparer les architectures logicielles réparties (Client/Serveur, Microservices, SOA, P2P).
- Comprendre les concepts du Cloud Computing (IaaS, PaaS, SaaS).
- Mettre en œuvre une architecture distribuée et déployée dans le Cloud.
- Découvrir les outils modernes : Docker, Kubernetes, Oracle Cloud.

Plan de formation : (Cours et TP)

Partie	**Titre**	**Contenu**
I	Introduction aux systèmes distribués	Définitions, caractéristiques, exemples réels, transparence, tolérance aux pannes
II	Modèles d'architecture	Client/serveur, SOA, Microservices, P2P, Middleware
III	Communication et coordination	Protocoles, RPC, horloges logiques, cohérence, réplication
IV	Sécurité et gestion distribuée	Authentification, certificats, logs, supervision
V	Introduction au Cloud Computing	Concepts, origines, principes NIST, acteurs
VI	Modèles de services (IaaS, PaaS, SaaS)	Définitions, comparaisons, cas d'usage
VII	Modèles de déploiement	Public, privé, hybride, multi-cloud
VIII	Technologies du Cloud	Virtualisation, Docker, Kubernetes, CI/CD
IX	Cloud-Native et microservices	12-Factor App, scalability, resilience

Partie 1 : Introduction aux systèmes distribués

Chapitre 1 – Concepts de base des systèmes distribués

1.Définitions :

Système distribué : ensemble de composants logiciels/hardwares situés sur différents ordinateurs, communiquant via un réseau et coopérant pour atteindre un but commun.

Voici quelques définitions académiques :

- Selon A. Tanenbaum, Distributed Systems: Principles and Paradigms, 2007) :

  « Un système distribué est un ensemble d'ordinateurs indépendants apparaissant à ses utilisateurs comme un système unique et cohérent. » En d'autres termes , plusieurs machines coopèrent pour offrir une illusion d'un seul système homogène.

- Selon ISO/IEC 7498-1 (modèle de référence OSI):

  « Un système distribué est un ensemble de systèmes autonomes interconnectés par un réseau de communication, coopérant pour fournir des services à des utilisateurs.»

- Selon ACM (Association for Computing Machinery)

  « Un système distribué est un ensemble de composants matériels et logiciels situés sur différents ordinateurs et communiquant par un réseau afin de coordonner leurs actions et atteindre un objectif commun. »

2.Motivations des architectures distribuées :

les systèmes distribués ont été inventés suite aux limites des anciennes architectures — en particulier les architectures centralisées (mainframes, client-serveur simple).

- Goulot d'étranglement (surcharge du serveur central)
- Faible fiabilité
- Manque de disponibilité
- Manque de flexibilité et d'évolutivité
- Faible partage des ressources
- Coût élevé et centralisation du contrôle
- Mauvaise tolérance géographique
- Faible coopération entre systèmes hétérogènes

3.Objectifs des architectures distribuées:

- **partage de ressources :** Le partage de ressources consiste à permettre à plusieurs utilisateurs ou applications, répartis sur différents ordinateurs, d'accéder aux mêmes ressources comme si elles étaient locales afin d'eviter la duplication inutile des ressources , d'optimiser leur utilisation et de réduire les coûts.
  Les ressources concernées peuvent être matérielles (imprimantes, disques, processeurs, etc.) ou logicielles (fichiers, bases de données, services, applications..)

  Exemple :
  Un serveur de fichiers accessible à tous les postes d'un réseau universitaire.

- **Scalabilité (ou extensibilité)** : La scalabilité est la capacité d'un système distribué à s'adapter à une augmentation de la charge (nombre d'utilisateurs, quantité de données, requêtes, etc.) sans dégradation notable des performances et sans nécessiter de refonte complète.. On distingue deux types de scalabilité , *horizontale* **(**ajout de nouvelles machines dans le réseau) et *verticale* (amélioration des ressources d'une machine (RAM, CPU, etc.)).

  Exemple :
  Ajouter des serveurs supplémentaires pour gérer le trafic accru d'un site web pendant les soldes.

- **Fiabilité (ou tolérance aux pannes)** : La fiabilité désigne la capacité d'un système distribué à *continuer à fonctionner correctement même en cas de panne partielle* (machine, réseau, processus, etc.) en évitant qu'une panne locale ne provoque l'arrêt global du système.
  Les mécanismes utilisés pour garantir la fiabilité sont :

    - Réplication des données et des services.
    - Détection et récupération automatique des pannes.
    - Redondance matérielle et logicielle.

  Exemple :
  Dans un cluster de serveurs, si un serveur tombe en panne, les autres prennent le relais automatiquement.

- **Transparence**: La transparence vise à masquer la complexité et la répartition physique du système aux utilisateurs et aux développeurs. Elle vise à simplifier l'utilisation et la gestion du système distribué.
  L'utilisateur doit interagir avec le système comme s'il s'agissait d'une seule machine.
  Il existe plusieurs types de transparence :

- Transparence d'accès : même façon d'accéder aux ressources locales ou distantes.
- Transparence de localisation : l'utilisateur n'a pas besoin de connaître où se trouve la ressource.
- Transparence de réplication : plusieurs copies d'une ressource apparaissent comme une seule.
- Transparence de panne : le système se rétablit sans intervention humaine.
- Transparence de migration : déplacement d'une ressource sans impacter l'utilisateur.

Exemple :
Un utilisateur ouvre un fichier partagé sans savoir sur quel serveur il est réellement stocké.

- **Disponibilité** : La disponibilité mesure la probabilité qu'un système soit opérationnel et accessible à un instant donné en assurant un service continu, même en cas de défaillance ou de maintenance.
  Les Moyens utilisés pour garantir la disponibilité :

  - Réplication des services.
  - Basculement automatique (failover).
  - Surveillance et équilibrage de charge (load balancing).

  Exemple :
  Un service de messagerie toujours accessible 24h/24, même pendant la maintenance d'un serveur.

4.Caractéristiques des systèmes distribués :

- **Concurrence des processus** : Dans un système distribué, plusieurs processus s'exécutent simultanément sur des machines différentes et coopèrent pour accomplir une tâche commune. Chaque machine exécute un ou plusieurs processus indépendants. Ces processus peuvent communiquer entre eux via un réseau (messages, RPC, sockets...).
  Comme ils s'exécutent en parallèle, il faut gérer :
  - les accès concurrents aux ressources partagées,
  - la synchronisation entre processus,
  - les conditions de concurrence (race conditions).

  Exemple :

  Deux serveurs de bases de données traitent en même temps des requêtes sur la même table. Il faut assurer la cohérence des données (verrous, transactions distribuées...).

- **Absence d'horloge globale** : Les ordinateurs d'un système distribué ne partagent pas une horloge commune. Chaque machine possède son propre temps local, qui peut dériver par rapport aux autres. Il n'existe pas de moyen simple pour garantir que tous les événements sont datés de manière cohérente sur l'ensemble des machines.
  Cette absence d'horloge commune complique :
  - la synchronisation temporelle (qui a fait quoi en premier ?),
  - la coordination des actions entre processus,
  - la gestion des messages et transactions.

  Exemple :
  Si deux serveurs envoient un message "presque" en même temps, leurs horloges locales peuvent donner des heures différentes. ↠ Difficile de déterminer l'ordre exact des événements sans un mécanisme spécial.

- **Tolérance aux pannes** : Un système distribué doit être capable de continuer à fonctionner correctement malgré la panne partielle d'un ou plusieurs de ses composants. Dans un réseau distribué, les pannes sont inévitables (serveur en panne, lien réseau coupé, perte de message...).
  Le système doit :
  - détecter la panne,
  - récupérer ou réassigner la tâche,
  - répliquer les données ou services pour ne rien perdre.

  Exemple :

  Dans un cluster web, si un serveur tombe, un autre prend automatiquement le relais (mécanisme de failover).

- **Hétérogénéité des ressources** : Les systèmes distribués intègrent des machines et des logiciels différents (systèmes d'exploitation, langages, architectures, protocoles...).
  Les composants peuvent être des serveurs Windows, Linux, macOS..., des terminaux mobiles, IoT, PC, serveurs cloud... ou des programmes écrits en Java, Python, C++...
  Le système doit assurer leur interopérabilité, c'est-à-dire leur capacité à communiquer d'une manière transparente malgré leurs différences.

  Exemple :

  Une application web peut faire appel à :
  - Une base de données Oracle sur Linux,
  - un microservice en Java sur un serveur Windows,
  - une API en Python sur le cloud AWS.

5.Exemples des systèmes distribués :

- Google Search, Netflix, Amazon Web Services, applications bancaires, jeux en ligne massifs.

6.Avantages et inconvénients des systèmes distribués :

Avantages	**Inconvénients**
Haute disponibilité Un système distribué reste opérationnel même en cas de panne d'un ou plusieurs composants, grâce à la redondance et à la réplication des services.	**Complexité accrue** La conception, le déploiement et la maintenance d'un système distribué sont beaucoup plus complexes qu'un système centralisé.
Scalabilité Capacité du système à s'adapter à une augmentation de la charge en ajoutant des ressources (machines, serveurs, processus...).	**Sécurité difficile** Plus il y a de nœuds et de communications sur un réseau, plus les risques de sécurité augmentent.
Répartition de charge La charge de travail est répartie entre plusieurs machines, évitant la saturation d'un seul point du système.	**Gestion d'erreurs complexe** Les défaillances partielles sont inévitables et difficiles à diagnostiquer dans un environnement distribué.

Chapitre 2 – Modèles d'architectures distribuées

(voir TP1)

1. Modèle Client/Serveur :
   - Clients demandent, serveur répond.
   - Exemple : application web classique (navigateur ↔ serveur)

2. Architecture à 3 tiers :
   - Présentation / Logique / Données
   - Exemple : front-end React, back-end Node.js, base MySQL.

3. SOA (Service-Oriented Architecture) :
   - Communication via services indépendants.
   - Utilise SOAP, WSDL, XML.

4. Microservices :
   - Petits services autonomes, interconnectés via API REST.
   - Communication asynchrone (HTTP, gRPC, message broker).

5. Architecture P2P (Peer-to-Peer) :
   - Chaque nœud joue le rôle de client et de serveur.
   - Exemple : BitTorrent, blockchain.

6. Middleware :
   - Logiciel intermédiaire facilitant la communication et la coordination entre services.
   - Exemples : CORBA, RMI, RabbitMQ, Kafka.

TP 1 : Découvrir les types d'architectures distribuées en Java

1-Objectifs :

- Identifier différents types d'architectures distribuées.
- Implémenter un mini-projet Client/Serveur et Microservices en Java.
- Comprendre la communication entre composants.

2-Configuation logicielle :

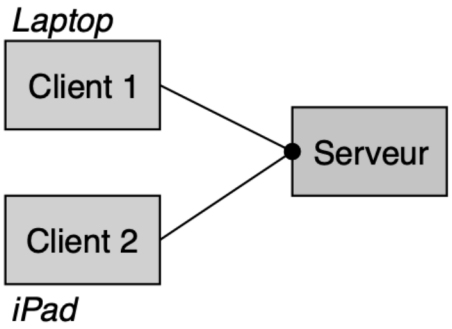
- JDK 17+ installé
- IDE Java (Eclipse, IntelliJ ou VS Code)
- Maven (pour microservices)
- Postman pour tester les services REST
- Connaissance de base en Java et en réseau

3-Travail demandé

- Compte rendu par binôme à remettre à la fin de chaque TP
- Chaque rapport contient : un résumé de chaque architectures, difficultés rencontrées, comparaisons.

Partie 1 : Architecture Client/Serveur

Objectif : Séparer un serveur et un client .



Exercice : Créer un serveur et un client simple en Java.

1-Serveur TCP Java :

```
import java.io.*;
import java.net.*;
public class Serveur {

   public static void main(String[] args) throws IOException {
      ServerSocket serverSocket = new ServerSocket(5000);
      System.out.println("Serveur lancé sur le port 5000...");
      Socket clientSocket = serverSocket.accept();

    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
                                BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));

      String message = in.readLine();
      System.out.println("Message reçu : " + message);
      out.println("Hello Client, votre message a été reçu !");
      clientSocket.close();
      serverSocket.close();
   }
}
```

2-Client TCP Java :

```
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket("localhost", 5000);

        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
                                    BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

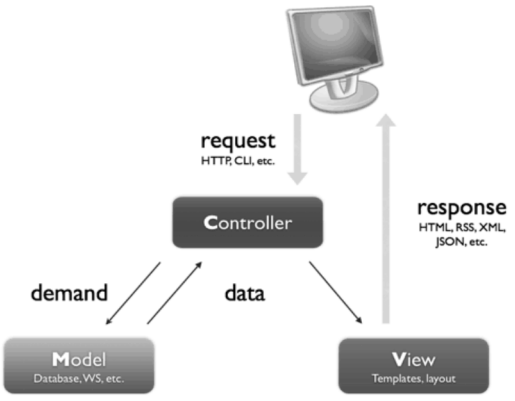
        out.println("Salut Serveur !");
        System.out.println("Réponse du serveur : " + in.readLine());
        socket.close();
    }
}
```

Questions :

- Que se passe-t-il si plusieurs clients se connectent en même temps ?
- Quel rôle jouent ServerSocket et Socket ?

Partie 2 : Architecture 3-Tiers

Objectif : Séparer la présentation, la logique métier et les données.



Exercice :

1. Créer un service étudiant (logique métier + données en mémoire) :

```
public class Etudiant {
   private int id;
   private String nom;
   // getters et setters
}
```

2. Créer un contrôleur qui expose un service REST (Spring Boot) :

```
@RestController
@RequestMapping("/etudiants")
public class EtudiantController {
   private List<Etudiant> etudiants = new ArrayList<>();

   @GetMapping
   public List<Etudiant> getEtudiants() {
      return etudiants;
   }

   @PostMapping
   public String addEtudiant(@RequestBody Etudiant e) {
      etudiants.add(e);
      return "Etudiant ajouté !";
   }
}
```

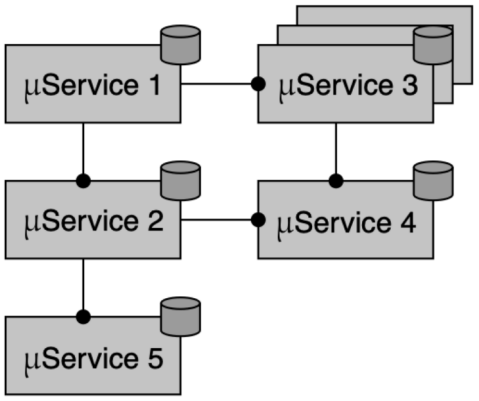
3. Créer une interface front simple (HTML + JavaScript fetch) pour afficher et ajouter des étudiants.

Questions :

- Quelles sont les responsabilités de chaque couche ?
- Pourquoi cette séparation améliorer la maintenabilité ?

Partie 3 : Microservices

Objectif : Implémenter des services indépendants



Exercice :

Implémenter deux services indépendants en Java (Spring Boot) :

1. Service Étudiants : CRUD sur les étudiants (port 8081)
2. Service Cours : CRUD sur les cours (port 8082)

Exercice :

- Chaque service est un Spring Boot project distinct.
- Communication via REST : le service Cours peut récupérer les étudiants du service Étudiants.

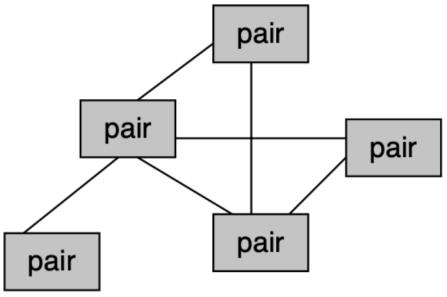
Questions :

- Comment gérer la communication si un service tombe en panne ?
- Quels avantages apporte la séparation en microservices par rapport à 3-Tiers ?

Partie 4 : Arhcitecture P2P

Objectif :

Simuler un réseau P2P .



Exercice :

Simuler un réseau P2P en Java.

- Chaque étudiant représente un “nœud” : un mini client-serveur Java.
- Ils échangent des messages directement sans serveur central.

Questions :

- Quels avantages/inconvénients par rapport au modèle client/serveur ?
- Donnez des exemples réels (BitTorrent, blockchain).

TP2 : API REST distribuée avec Java

1- Contexte

Implémenter une application distribuée composée de deux services REST indépendants :

- ServiceEtudiants (CRUD des étudiants) — port 8081
- ServiceCours (CRUD des cours, associe cours ↠ étudiant via etudiantId) — port 8082
- structure d'un projet Spring Boot + Maven
- création d'API REST (GET, POST, PUT, DELETE)

2-Objectifs

1. Créer des microservices Spring Boot autonomes.
2. Définir et documenter des endpoints REST (JSON).
3. Implémenter la communication entre services via HTTP.
4. Conteneuriser les services et les démarrer via docker-compose.
5. Tester et démontrer le fonctionnement des interactions (Postman).

3-Environnement du travail

- JDK 17+
- Maven 3+
- Docker & Docker Compose (pour la partie conteneurisation)
- IDE (IntelliJ/Eclipse/VS Code)
- Postman (ou curl)

4- Livrables attendus

- Deux projets Maven (ServiceEtudiants, ServiceCours) avec code.
- docker-compose.yml et Dockerfile pour chaque service.
- Collection Postman (JSON) ou fichier requests.http.
- Rapport court (1 page) : architecture et trois captures d'écran (GET /etudiants, GET /cours, interaction inter-service).

Partie A : ServiceEtudiants

1- Structure du projet

```
ServiceEtudiants/
 ├── src/main/java/fr/univ/tp/etudiants/
 |    ├── ServiceEtudiantsApplication.java
 |    ├── model/Etudiant.java
 |    ├── controller/EtudiantController.java
 |    └── repository/EtudiantRepository.java   (in-memory)
 └── pom.xml
```

2- Fichier POM.XML(minimal)

```
<project ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>fr.univ.tp</groupId>
  <artifactId>ServiceEtudiants</artifactId>
  <version>1.0.0</version>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.1.0</version>
  </parent>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
</project>
```

3- ServiceEtudiantsApplication.java

```
package fr.univ.tp.etudiants;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class ServiceEtudiantsApplication {
    public static void main(String[] args) {
        SpringApplication.run(ServiceEtudiantsApplication.class, args);
    }
}
```

4-model/Etudiant.java

```
package fr.univ.tp.etudiants.model;

public class Etudiant {
    private Integer id;
    private String nom;
    private String email;

    public Etudiant() {}
    public Etudiant(Integer id, String nom, String email) {
        this.id = id; this.nom = nom; this.email = email;
    }
    // getters & setters
}
```

5- repository/EtudiantRepository.java (in-memory simple)

```
package fr.univ.tp.etudiants.repository;

import fr.univ.tp.etudiants.model.Etudiant;
import org.springframework.stereotype.Repository;
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

@Repository
public class EtudiantRepository {
    private final Map<Integer, Etudiant> store = new HashMap<>();
    private final AtomicInteger seq = new AtomicInteger(1);

    public List<Etudiant> findAll() { return new ArrayList<>(store.values()); }
    public Optional<Etudiant> findById(int id) { return Optional.ofNullable(store.get(id)); }

    public Etudiant save(Etudiant e) {
        if (e.getId() == null) e.setId(seq.getAndIncrement());
        store.put(e.getId(), e);
        return e;
    }
    public void delete(int id) { store.remove(id); }
}
```

6-controller/EtudiantController.java

```
package fr.univ.tp.etudiants.controller;

import fr.univ.tp.etudiants.model.Etudiant;
import fr.univ.tp.etudiants.repository.EtudiantRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/etudiants")
public class EtudiantController {
   private final EtudiantRepository repo;
   public EtudiantController(EtudiantRepository repo) { this.repo = repo; }

   @GetMapping
   public List<Etudiant> all() { return repo.findAll(); }

   @GetMapping("/{id}")
   public Etudiant get(@PathVariable int id) {
          return repo.findById(id).orElseThrow(() -> new RuntimeException("Etudiant non
trouvé"));
   }

   @PostMapping
   public Etudiant create(@RequestBody Etudiant e) { return repo.save(e); }

   @DeleteMapping("/{id}")
   public void delete(@PathVariable int id) { repo.delete(id); }
}
```

7-Exécution

Lancer

- mvn spring-boot:run (ou exécuter la classe ServiceEtudiantsApplication)
- Endpoints :
    - GET http://localhost:8081/etudiants
    - POST http://localhost:8081/etudiants avec JSON { "nom":"Ali", "email":"ali@univ.com" }

Partie B : ServiceCours

1- Structure du projet

```
ServiceCours/
 ├── src/main/java/fr/univ/tp/cours/
 |    ├── ServiceCoursApplication.java
 |    ├── model/Cours.java
 |    ├── controller/CoursController.java
 |    └── repository/CoursRepository.java   (in-memory)
 └── pom.xml
```

2- Fichier pom.xml (identique dépendances web)

(pareil, groupId/artifactId différents)

3- model/Cours.java

```
package fr.univ.tp.cours.model;

public class Cours {
   private Integer id;
   private String titre;
   private Integer etudiantId; // lien vers Etudiant
   public Cours() {}
   public Cours(Integer id, String titre, Integer etudiantId) {
      this.id = id; this.titre = titre; this.etudiantId = etudiantId;
   }
   // getters & setters
}
```

4- repository/CoursRepository.java : (same pattern as EtudiantRepository)

5- controller/CoursController.java

```
package fr.univ.tp.cours.controller;

import fr.univ.tp.cours.model.Cours;
import fr.univ.tp.cours.repository.CoursRepository;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/cours")
public class CoursController {
   private final CoursRepository repo;
   public CoursController(CoursRepository repo) { this.repo = repo; }

   @GetMapping
   public List<Cours> all() { return repo.findAll(); }

   @GetMapping("/{id}")
   public Cours get(@PathVariable int id) {
            return repo.findById(id).orElseThrow(() -> new RuntimeException("Cours non
trouvé"));
   }

   @PostMapping
   public Cours create(@RequestBody Cours c) { return repo.save(c); }

   @DeleteMapping("/{id}")
   public void delete(@PathVariable int id) { repo.delete(id); }
}
```

6-Exécution

Lancer

- Lancer sur le port 8082 : dans application.properties, ajouter server.port=8082
- Endpoints : GET http://localhost:8082/cours, POST http://localhost:8082/cours

Partie C : Communication inter-services

Objectif :

- ServiceCours récupère les données d'un étudiant depuis ServiceEtudiants et renvoie une vue enrichie : endpoint GET /cours/enrichis qui retourne la liste des cours avec le nom de l'étudiant associé.
- Approche simple : RestTemplate (synchronique)
- Ajouter dans ServiceCours dépendance Spring Web (déjà présente).

Exemple d'utilisation :

1- service/EtudiantClient.java

```
package fr.univ.tp.cours.service;

import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import fr.univ.tp.etudiants.model.Etudiant; // pour DTO commun ou créer une classe DTO locale

@Service
public class EtudiantClient {
    private final RestTemplate rest = new RestTemplate();
    private final String baseUrl = "http://localhost:8081/etudiants";

    public Etudiant getEtudiantById(int id) {
        return rest.getForObject(baseUrl + "/" + id, Etudiant.class);
    }
}
```

Remarque :

Si on ne souhaite pas importer Etudiant du projet ServiceEtudiants, créer un DTO EtudiantDto identique dans ServiceCours.

2- controller/CoursController.java (méthode enrichie)

```
@GetMapping("/enrichis")
public List<Map<String,Object>> getCoursEnrichis() {
   List<Cours> cours = repo.findAll();
   List<Map<String,Object>> result = new ArrayList<>();
   for (Cours c : cours) {
      Map<String,Object> m = new HashMap<>();
      m.put("id", c.getId());
      m.put("titre", c.getTitre());
      try {
         Etudiant et = etudiantClient.getEtudiantById(c.getEtudiantId());
         m.put("etudiantNom", et.getNom());
      } catch (Exception ex) {
         m.put("etudiantNom", "INCONNU");
      }
      result.add(m);
   }
   return result;
}
```

3- Tests à effectuer (Postman)

1. POST étudiant sur :8081/etudiants — créer {"nom":"Sara","email":"sara@u.com"}
2. POST cours sur :8082/cours — créer {"titre":"Prog 3","etudiantId":1}
3. GET :8082/cours/enrichis ↠ doit retourner cours + etudiantNom: "Sara"

Questions :

- Que se passe-t-il si ServiceEtudiants est indisponible ? (simulatez en arrêtant le service)
- Proposez une solution pour rendre la communication plus résiliente.

Partie D : Tests et scénarios

Préparer une collection Postman (ou fichiers cURL) avec ces scénarios :

1. Création et lecture d'étudiant (POST, GET)
2. Création et lecture de cours (POST, GET)
3. Interaction enrichie (GET /cours/enrichis)
4. Test de suppression d'étudiant et effet sur /cours/enrichis
5. Simuler panne : arrêter ServiceEtudiants et appeler /cours/enrichis ↠ gérer erreur

Vérifications attendues :

- Structure JSON conforme
- Codes HTTP appropriés (200, 201, 404, 500 selon cas)
- Comportement en cas d'erreur (message lisible)

Partie E : Dockerisation & docker-compose

1- Fournir Dockerfile simple pour chaque service.

```
Dockerfile (ex. ServiceEtudiants)
FROM eclipse-temurin:17-jdk-jammy
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

2- Build & run local :

- mvn clean package
- docker build -t serviceetudiants:1.0 .
- docker run -p 8081:8081 serviceetudiants:1.0

```
docker-compose.yml
version: '3.8'
services:
  etudiants:
    build: ./ServiceEtudiants
    ports:
      - "8081:8081"
  cours:
    build: ./ServiceCours
    ports:
      - "8082:8082"
    depends_on:
      - etudiants
```

3- Commandes utiles :

- docker-compose up --build
- Accéder aux endpoints comme précédemment (les services seront accessibles en localhost).

4- Travail à rendre :

- Le code source pour chaque service (zip ou dépôt Git).
- Collection Postman exportée (.json) ou requests.http.
- Rapport synthétique 1 page + 3 captures (GET /etudiants, GET /cours, GET /cours/enrichis).